

ThoughtWorks®

using & building

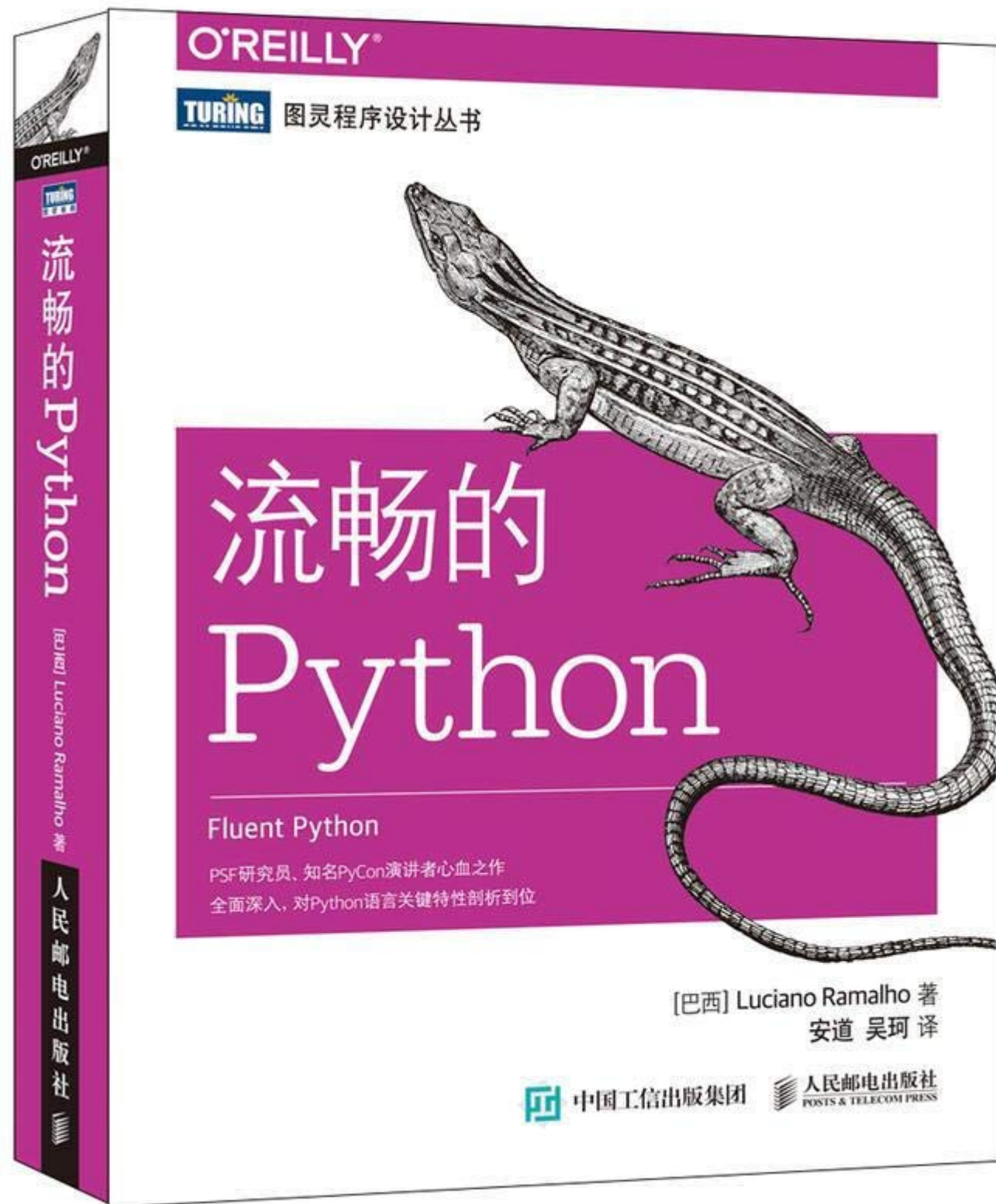
PYTHON SET PRACTICE

Learn great API design ideas from Python's set types.



Luciano Ramalho
@standupdev

FLUENT PYTHON



Available in 9 languages:

- Chinese (simplified)
- Chinese (traditional)
- English
- French
- Russian
- Japanese
- Korean
- Polish
- Portuguese

2nd ed: I'm working on it!

AGENDA

Motivation

Overview of Python Sets

Learning from the set API

The **__magic__** behind a set class

MOTIVATION

Some common use cases for sets

USE CASE #1

**display product if all
words in the query appear
in the product
description.**

HAND-ROLLED SOLUTION #1

I've written code like this in Go, which lacks built-in sets:

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        found := false  
        for _, elem := range slice {  
            if needle == elem {  
                found = true  
                break  
            }  
        }  
        if !found {  
            return false  
        }  
    }  
    return true  
}
```

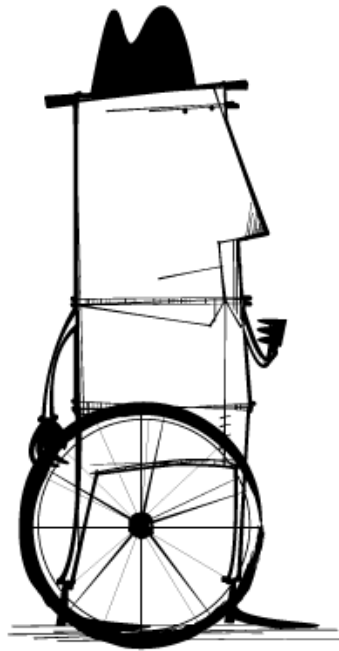
HAND-ROLLED SOLUTION #2

More readable, but still inefficient:

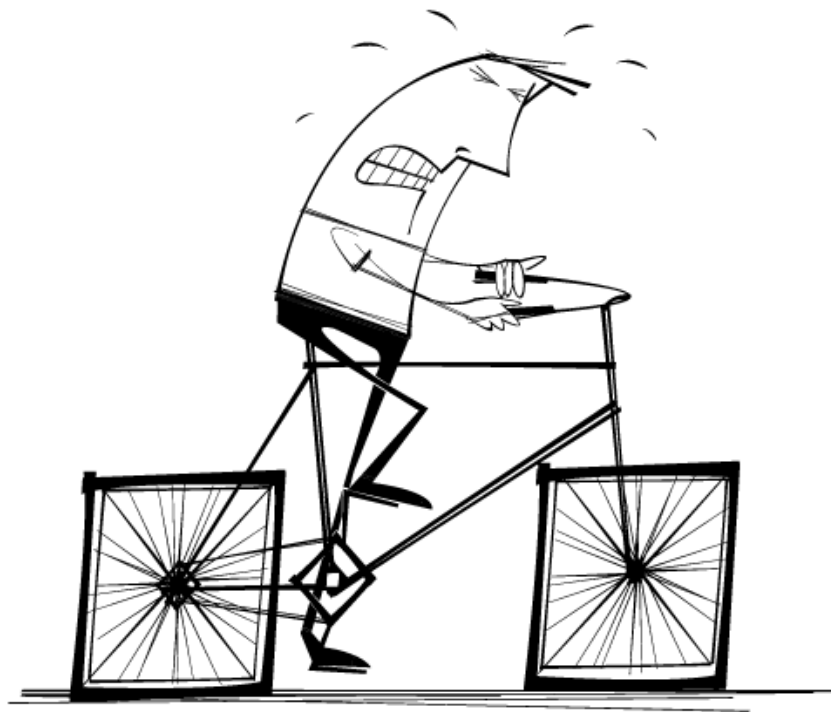
```
func Contains(slice []string, needle string) bool {  
    for _, elem := range slice {  
        if needle == elem {  
            return true  
        }  
    }  
    return false  
}
```

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        if !Contains(slice, needle) {  
            return false  
        }  
    }  
    return true  
}
```

What if...



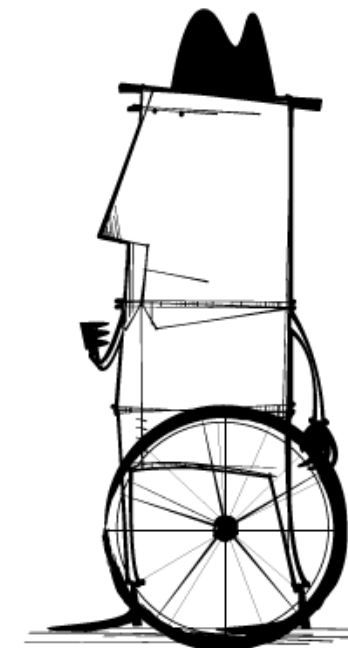
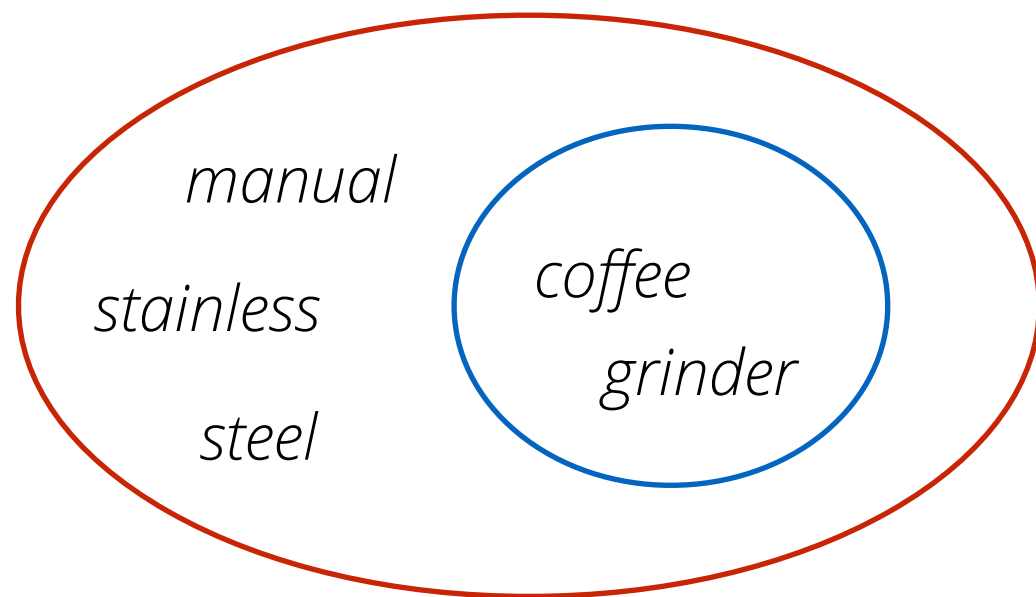
Later! I am too busy
coding nested loops!



USE CASE #1

www.workcompass.com/

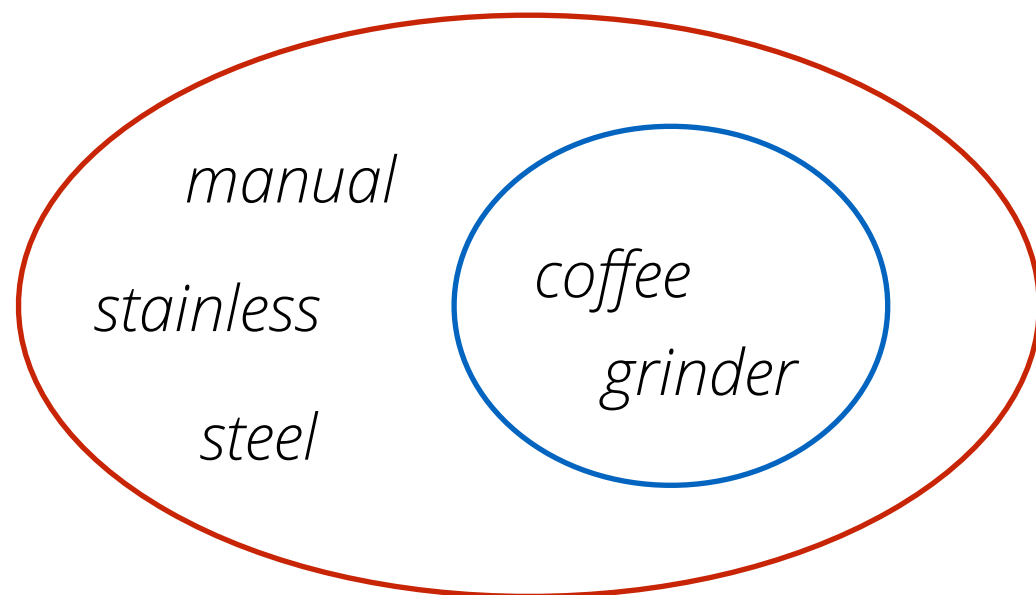
**display product if all
words in the query appear
in the product
description.**



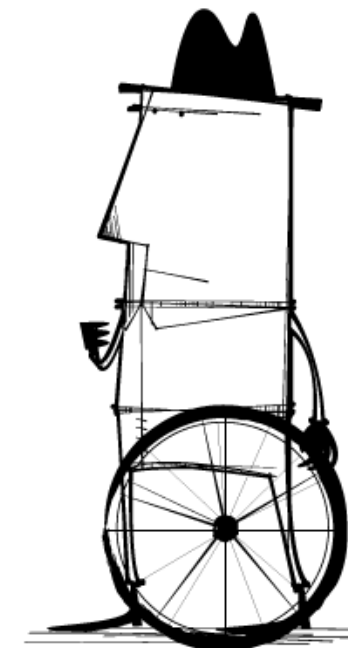
USE CASE #1

www.workcompass.com/

**display product if all
words in the query appear
in the product
description.**



$$Q \subset D$$

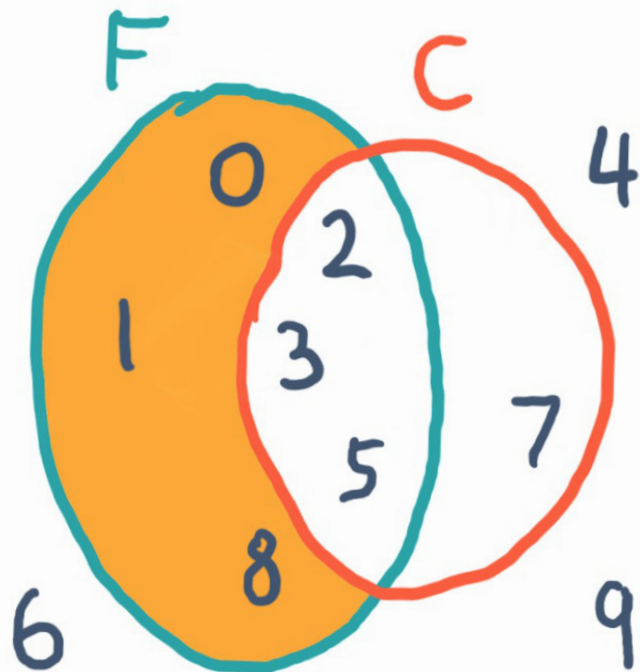


USE CASE #2

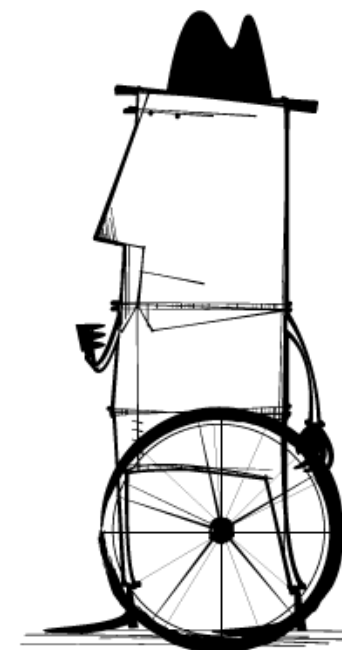
Mark all products previously favorited, except those already in the shopping cart.

USE CASE #2

Mark all products previously favorited, except those already in the shopping cart.



$$F \setminus C$$



LOGIC AND SETS

A close relationship

Nobody has yet discovered a branch of mathematics that has successfully resisted formalization into set theory.

Thomas Forster
Logic Induction and Sets, p. 167

LOGIC CONJUNCTION IS INTERSECTION

x belongs to the intersection of A with B.

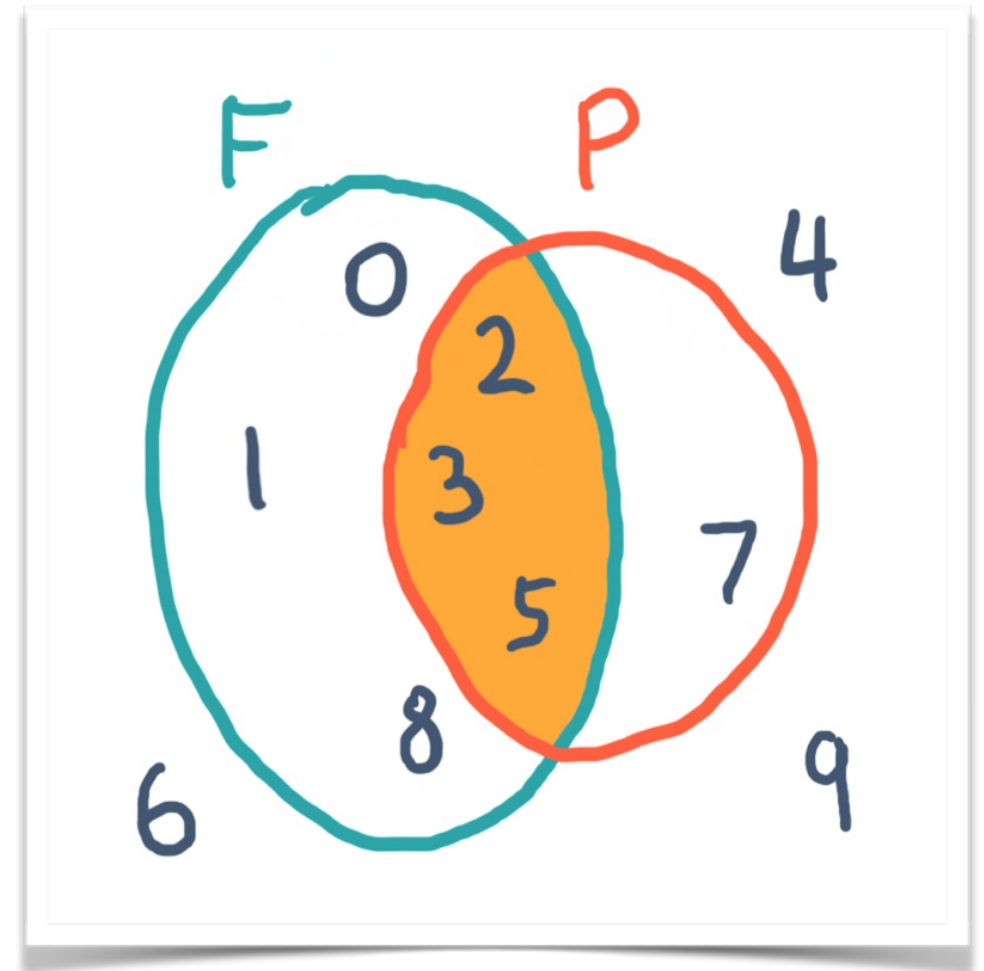
is the same as:

*x belongs to A **and**
x also belongs to B.*

Math notation:

$$x \in (A \cap B) \iff (x \in A) \wedge (x \in B)$$

In computing: **AND**



LOGIC DISJUNCTION: UNION

x belongs to the union of A and B.

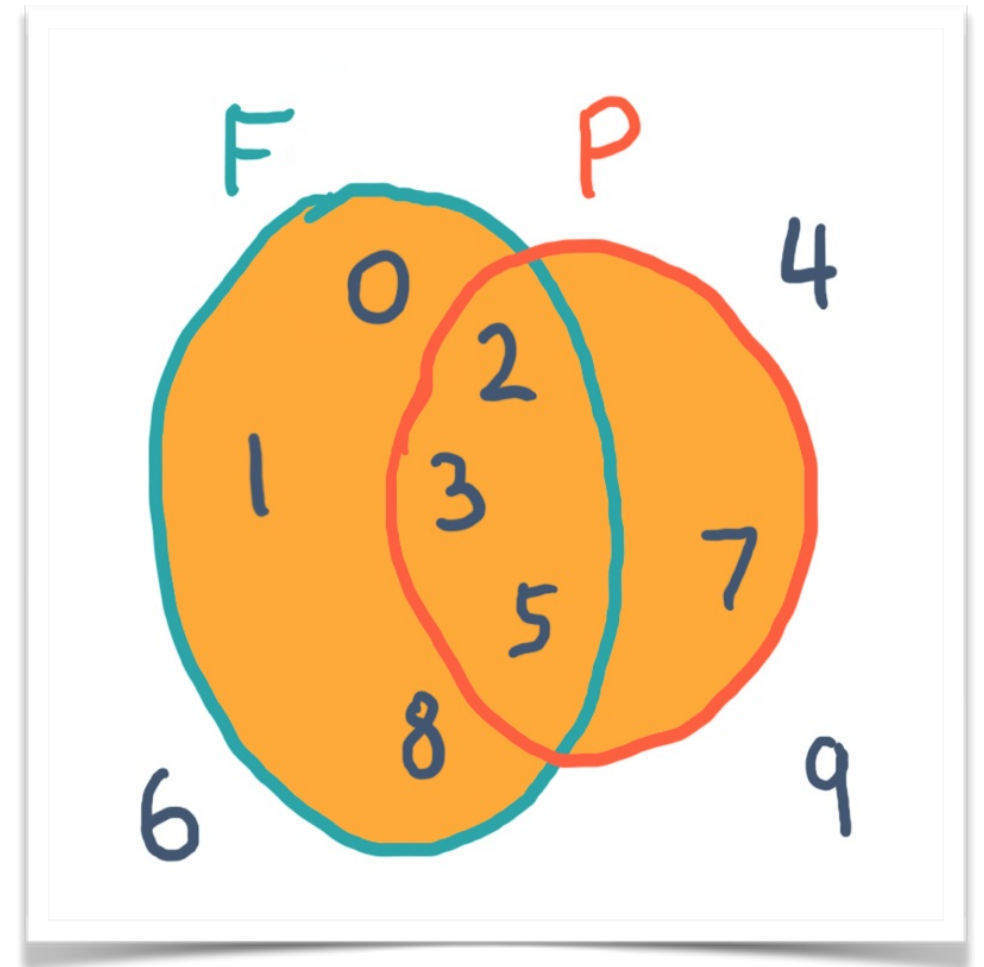
is the same as:

*x belongs to A **or**
x belongs to B.*

Math notation:

$$x \in (A \cup B) \iff (x \in A) \vee (x \in B)$$

In computing: **OR**



SYMMETRIC DIFFERENCE

*x belongs to A **or**
 x belongs to B but
does not belong to both*

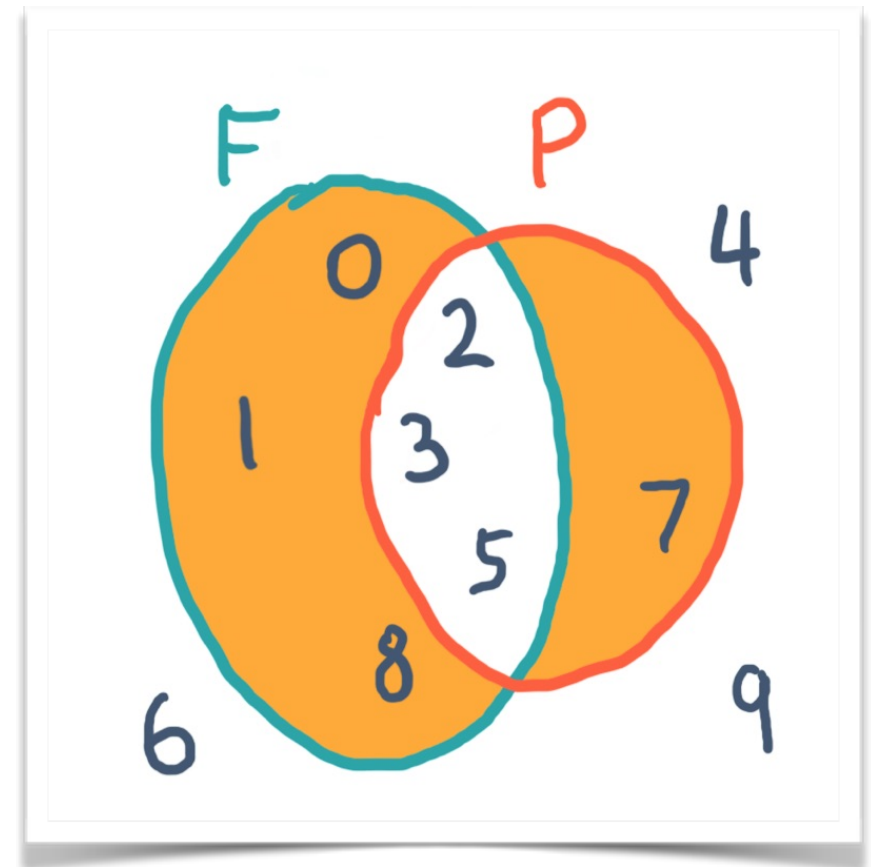
Is the same as:

*x belongs to the union of A with B
less the intersection of A with B .*

Math notation:

$$x \in (A \Delta B) \iff (x \in A) \vee (x \in B)$$

In computing: **XOR**

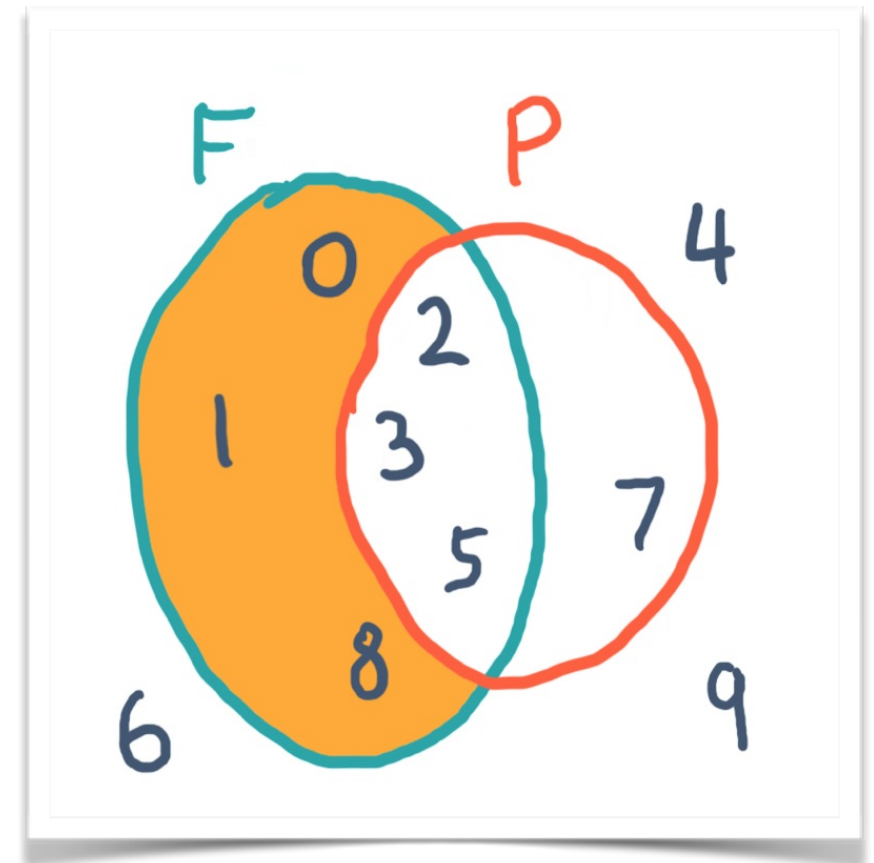


DIFFERENCE

*x belongs to A but
does not belong to B .*

is the same as:

*elements of A minus elements of
 B*



Math notation:

$$x \in (A \setminus B) \iff (x \in A) \wedge (x \notin B)$$

SETS IN SEVERAL LANGUAGES

SETS IN SEVERAL STANDARD LIBRARIES

Some languages/platform APIs that implement sets in their standard libraries

Java **Set** interface: < 10 methods; 8 implementations

Python **set, frozenset**: > 10 methods and operators

.Net (C# etc.) **ISet** interface: > 10 methods; 2 implementations

JavaScript (ES6) **Set**: < 10 methods

Ruby **Set**: > 10 methods and operators

Python, .Net and Ruby offer rich set APIs

SETS IN PYTHON

The built-in types

BUILDING A SET FROM A SERIES OF NUMBERS

Using a set comprehension:

```
In [1]: 1 def fibonacci(stop):  
        2     a, b = 0, 1  
        3     while a < stop:  
        4         yield a  
        5         a, b = b, a + b
```

```
In [2]: 1 f = {n for n in fibonacci(10)}  
        2 f
```

```
Out[2]: {0, 1, 2, 3, 5, 8}
```

ANOTHER SET, FOR THE EXAMPLES

In [3]:

```
1 def primes(stop):
2     '''Yields the sequence of prime numbers via
3     m = {} # map composite integers to primes
4     q = 2  # first integer to test for primality
5     while q < stop:
6         if q not in m:
7             yield q          # not marked composite
8             m[q*q] = [q]     # first multiple of q
9         else:
10            for p in m[q]: # move each witness
11                m.setdefault(p+q, []).append(p)
12            del m[q]         # no longer need m[q]
13            q += 1
```

In [4]:

```
1 p = {n for n in primes(10)}
2 p
```

Out[4]: {2, 3, 5, 7}

ELEMENT CONTAINMENT: THE IN OPERATOR

$O(1)$ in sets, because they use a hash table to hold elements.

Implemented by the **`__contains__`** special method:

```
In [6]: 1 1 in f
```

```
Out[6]: True
```

```
In [7]: 1 1 in p
```

```
Out[7]: False
```

FUNDAMENTAL SET OPERATIONS

```
In [8]: 1 f & p
```

Intersection

```
Out[8]: {2, 3, 5}
```

```
In [9]: 1 f | p
```

Union

```
Out[9]: {0, 1, 2, 3, 5, 7, 8}
```

```
In [10]: 1 f ^ p
```

Symmetric difference (a.k.a. XOR)

```
Out[10]: {0, 1, 7, 8}
```

```
In [11]: 1 f - p
```

Difference

```
Out[11]: {0, 1, 8}
```

```
In [12]: 1 p - f
```

```
Out[12]: {7}
```

SET COMPARISONS

Subset and superset testing.

In math: $\subset$, $\subseteq$, $\supset$, $\supseteq$.

```
In [13]: 1 f >= p
```

```
Out[13]: False
```

```
In [14]: 1 p >= f
```

```
Out[14]: False
```

```
In [15]: 1 f >= {1, 2, 3}
```

```
Out[15]: True
```

```
In [16]: 1 p >= {1, 2, 3}
```

```
Out[16]: False
```

DE MORGAN'S LAW: #1

```
In [17]: 1 e = {n for n in range(10) if n % 2 == 0}
          2 e
```

```
Out[17]: {0, 2, 4, 6, 8}
```

```
In [18]: 1 p & e
```

```
Out[18]: {2}
```

```
In [19]: 1 f - (p & e)
```

```
Out[19]: {0, 1, 3, 5, 8}
```

```
In [20]: 1 f - (p & e) == (f - p) | (f - e)
```

```
Out[20]: True
```

DE MORGAN'S LAW: #2

```
In [21]: 1 p | e
```

```
Out[21]: {0, 2, 3, 4, 5, 6, 7, 8}
```

```
In [22]: 1 f - (p | e)
```

```
Out[22]: {1}
```

```
In [23]: 1 f - (p | e) == (f - p) & (f - e)
```

```
Out[23]: True
```

SET METHODS

Going beyond what operators can do.

SET OPERATORS AND METHODS (1)

Table 3-2. Mathematical set operations: these methods either produce a new set or update the target set in place, if it's mutable

Math symbol	Python operator	Method	Description
$S \cap Z$	<code>s & z</code>	<code>s.__and__(z)</code>	Intersection of <code>s</code> and <code>z</code>
	<code>z & s</code>	<code>s.__rand__(z)</code>	Reversed <code>&</code> operator
		<code>s.intersection(it, ...)</code>	Intersection of <code>s</code> and all sets built from iterables <code>it</code> , etc.
	<code>s &= z</code>	<code>s.__iand__(z)</code>	<code>s</code> updated with intersection of <code>s</code> and <code>z</code>
		<code>s.intersection_update(it, ...)</code>	<code>s</code> updated with intersection of <code>s</code> and all sets built from iterables <code>it</code> , etc.
$S \cup Z$	<code>s z</code>	<code>s.__or__(z)</code>	Union of <code>s</code> and <code>z</code>
	<code>z s</code>	<code>s.__ror__(z)</code>	Reversed <code> </code>
		<code>s.union(it, ...)</code>	Union of <code>s</code> and all sets built from iterables <code>it</code> , etc.
	<code>s = z</code>	<code>s.__ior__(z)</code>	<code>s</code> updated with union of <code>s</code> and <code>z</code>
		<code>s.update(it, ...)</code>	<code>s</code> updated with union of <code>s</code> and all sets built from iterables <code>it</code> , etc.

SET OPERATORS AND METHODS (2)

Differences:

$S \setminus Z$	$s - z$	<code>s.__sub__(z)</code>	Relative complement or difference between <code>s</code> and <code>z</code>
	$z - s$	<code>s.__rsub__(z)</code>	Reversed - operator
		<code>s.difference(it, ...)</code>	Difference between <code>s</code> and all sets built from iterables <code>it</code> , etc.
	$s -= z$	<code>s.__isub__(z)</code>	<code>s</code> updated with difference between <code>s</code> and <code>z</code>
		<code>s.difference_update(it, ...)</code>	<code>s</code> updated with difference between <code>s</code> and all sets built from iterables <code>it</code> , etc.
		<code>s.symmetric_difference(it)</code>	Complement of <code>s & set(it)</code>
$S \Delta Z$	$s \wedge z$	<code>s.__xor__(z)</code>	Symmetric difference (the complement of the intersection <code>s & z</code>)
	$z \wedge s$	<code>s.__rxor__(z)</code>	Reversed $\wedge$ operator
		<code>s.symmetric_difference_update(it, ...)</code>	<code>s</code> updated with symmetric difference of <code>s</code> and all sets built from iterables <code>it</code> , etc.
	$s \wedge= z$	<code>s.__ixor__(z)</code>	<code>s</code> updated with symmetric difference of <code>s</code> and <code>z</code>

SET TESTS

All of these return a bool:

Table 3-3. Set comparison operators and methods that return a bool

Math symbol	Python operator	Method	Description
		<code>s.isdisjoint(z)</code>	s and z are disjoint (have no elements in common)
$e \in S$	<code>e in s</code>	<code>s.__contains__(e)</code>	Element e is a member of s
$S \subseteq Z$	<code>s <= z</code>	<code>s.__le__(z)</code>	s is a subset of the z set
		<code>s.issubset(it)</code>	s is a subset of the set built from the iterable it
$S \subset Z$	<code>s < z</code>	<code>s.__lt__(z)</code>	s is a proper subset of the z set
$S \supseteq Z$	<code>s >= z</code>	<code>s.__ge__(z)</code>	s is a superset of the z set
		<code>s.issuperset(it)</code>	s is a superset of the set built from the iterable it
$S \supset Z$	<code>s > z</code>	<code>s.__gt__(z)</code>	s is a proper superset of the z set

ADDITIONAL METHODS

These have nothing to do with math, and all to do with practical computing:

Table 3-4. Additional set methods

	set	frozenset	
<code>s.add(e)</code>	•		Add element <code>e</code> to <code>s</code>
<code>s.clear()</code>	•		Remove all elements of <code>s</code>
<code>s.copy()</code>	•	•	Shallow copy of <code>s</code>
<code>s.discard(e)</code>	•		Remove element <code>e</code> from <code>s</code> if it is present
<code>s.__iter__()</code>	•	•	Get iterator over <code>s</code>
<code>s.__len__()</code>	•	•	<code>len(s)</code>
<code>s.pop()</code>	•		Remove and return an element from <code>s</code> , raising <code>KeyError</code> if <code>s</code> is empty
<code>s.remove(e)</code>	•		Remove element <code>e</code> from <code>s</code> , raising <code>KeyError</code> if <code>e not in s</code>

ABSTRACT SET INTERFACES

These interfaces are all defined in **collections.abc**.
set and **frozenset** both implement **Set**
set also implements **MutableSet**

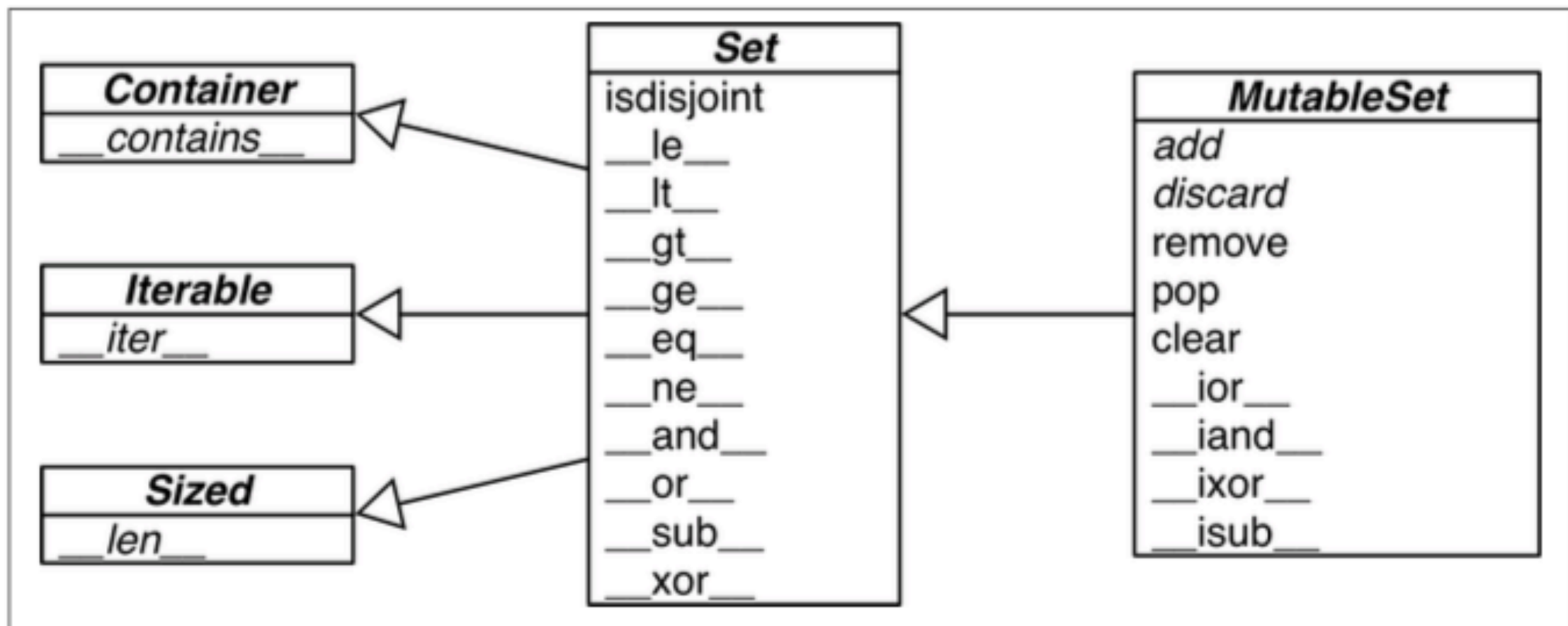


Figure 3-2. UML class diagram for *MutableSet* and its superclasses from *collections.abc* (names in italics are abstract classes and abstract methods; reverse operator methods omitted for brevity)

OPERATOR OVERLOADING

Not as bad as they say

COMPARISON OPERATORS

Table 13-2. Rich comparison operators: reverse methods invoked when the initial method call returns NotImplemented

Group	Infix operator	Forward method call	Reverse method call	Fall back
Equality	<code>a == b</code>	<code>a.__eq__(b)</code>	<code>b.__eq__(a)</code>	Return <code>id(a) == id(b)</code>
	<code>a != b</code>	<code>a.__ne__(b)</code>	<code>b.__ne__(a)</code>	Return <code>not (a == b)</code>
Ordering	<code>a > b</code>	<code>a.__gt__(b)</code>	<code>b.__lt__(a)</code>	Raise <code>TypeError</code>
	<code>a < b</code>	<code>a.__lt__(b)</code>	<code>b.__gt__(a)</code>	Raise <code>TypeError</code>
	<code>a >= b</code>	<code>a.__ge__(b)</code>	<code>b.__le__(a)</code>	Raise <code>TypeError</code>
	<code>a <= b</code>	<code>a.__le__(b)</code>	<code>b.__ge__(a)</code>	Raise <code>TypeError</code>

Table 13-1. Infix operator method names (the in-place operators are used for augmented assignment; comparison operators are in *Table 13-2*)

Operator	Forward	Reverse	In-place	Description
+	<code>__add__</code>	<code>__radd__</code>	<code>__iadd__</code>	Addition or concatenation
-	<code>__sub__</code>	<code>__rsub__</code>	<code>__isub__</code>	Subtraction
*	<code>__mul__</code>	<code>__rmul__</code>	<code>__imul__</code>	Multiplication or repetition
/	<code>__truediv__</code>	<code>__rtruediv__</code>	<code>__itruediv__</code>	True division
//	<code>__floordiv__</code>	<code>__rfloordiv__</code>	<code>__ifloordiv__</code>	Floor division
%	<code>__mod__</code>	<code>__rmod__</code>	<code>__imod__</code>	Modulo
<code>divmod()</code>	<code>__divmod__</code>	<code>__rdivmod__</code>	<code>__idivmod__</code>	Returns tuple of floor division quotient and modulo
<code>**</code> , <code>pow()</code>	<code>__pow__</code>	<code>__rpow__</code>	<code>__ipow__</code>	Exponentiation ^a
@	<code>__matmul__</code>	<code>__rmatmul__</code>	<code>__imatmul__</code>	Matrix multiplication ^b
&	<code>__and__</code>	<code>__rand__</code>	<code>__iand__</code>	Bitwise and
	<code>__or__</code>	<code>__ror__</code>	<code>__ior__</code>	Bitwise or
^	<code>__xor__</code>	<code>__rxor__</code>	<code>__ixor__</code>	Bitwise xor
<<	<code>__lshift__</code>	<code>__rlshift__</code>	<code>__ilshift__</code>	Bitwise shift left
>>	<code>__rshift__</code>	<code>__rrshift__</code>	<code>__irshift__</code>	Bitwise shift right

^a `pow` takes an optional third argument, `modulo`: `pow(a, b, modulo)`, also supported by the special methods when invoked directly (e.g., `a.__pow__(b, modulo)`).

^b New in Python 3.5.

THE BEAUTY OF DOUBLE DISPATCH

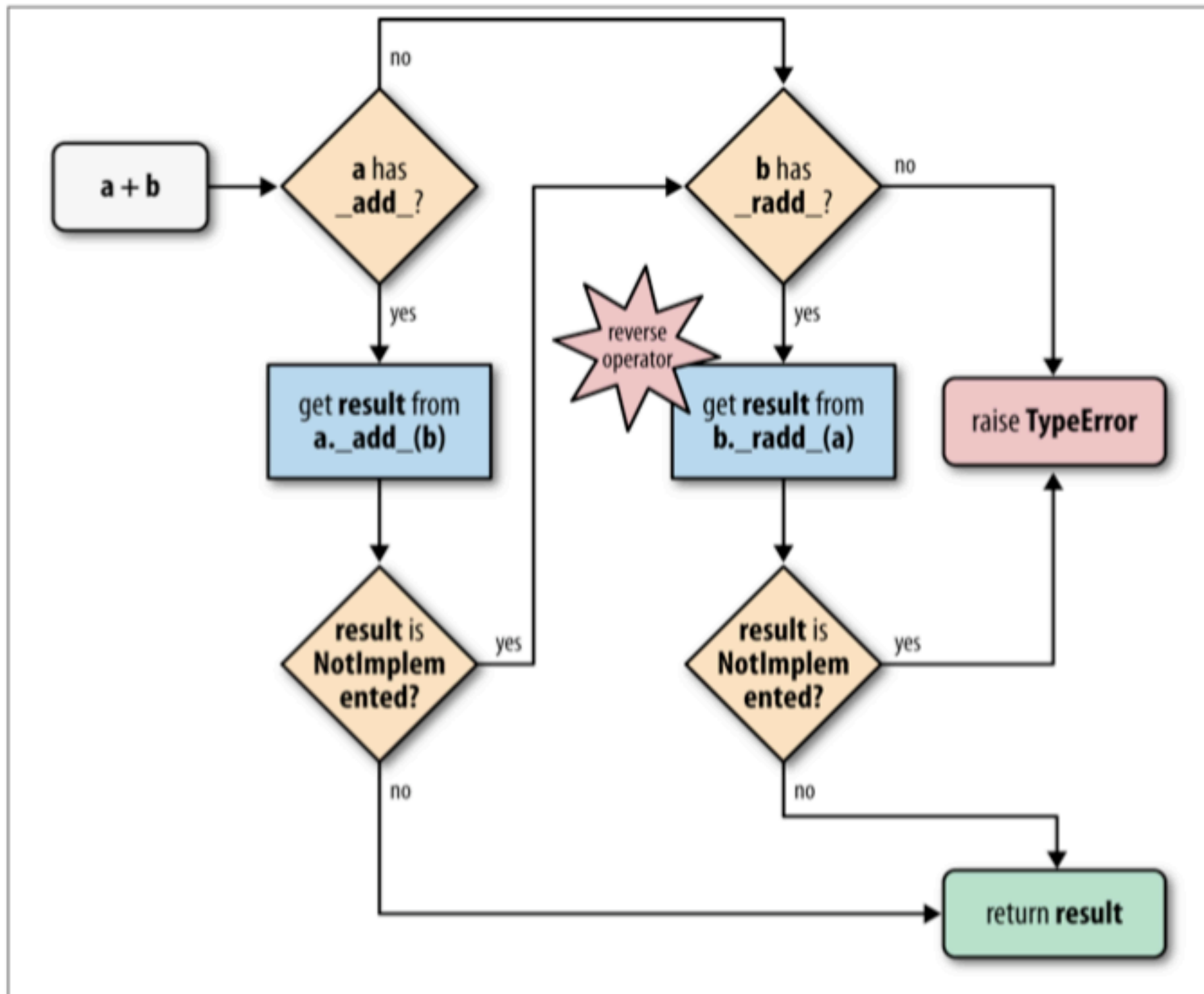


Figure 13-1. Flowchart for computing $a + b$ with `__add__` and `__radd__`

EXAMPLE IMPLEMENTATION

A set for non-negative integers

UINTSET: A SET CLASS FOR NON-NEGATIVE INTEGERS

Inspired by the **intset** example in chapter 6 of *The Go Programming Language* by **A. Donovan** and **B. Kernighan**

An empty set is represented by zero.

A set of integers {a, b, c} is represented by on bits in an integer at offsets a, b, and c.

Source code:

<https://github.com/standupdev/uintset>

REPRESENTING SETS OF INTEGERS AS BIT PATTERNS

This set:

```
UIntSet({13, 14, 22, 28, 38, 53, 64, 76, 94, 102, 107, 121,  
136, 143, 150, 157, 169, 173, 187, 201, 213, 216, 234, 247,  
257, 268, 283, 288, 290})
```

REPRESENTING SETS OF INTEGERS AS BIT PATTERNS

This set:

```
UIntSet({13, 14, 22, 28, 38, 53, 64, 76, 94, 102, 107, 121,  
136, 143, 150, 157, 169, 173, 187, 201, 213, 216, 234, 247,  
257, 268, 283, 288, 290})
```

Is represented by this integer

```
2502158007702946921897431281681230116680925854234644385938703  
363396454971897652283727872
```

REPRESENTING SETS OF INTEGERS AS BIT PATTERNS

This set:

```
UIntSet({13, 14, 22, 28, 38, 53, 64, 76, 94, 102, 107, 121,  
136, 143, 150, 157, 169, 173, 187, 201, 213, 216, 234, 247,  
257, 268, 283, 288, 290})
```

Is represented by this integer

```
2502158007702946921897431281681230116680925854234644385938703  
363396454971897652283727872
```

Which has this bit pattern:

```
10100001000000000000000010000000000010000000000100000000000010000  
000000000000001001000000000000100000000000001000000000000010001  
00000000000010000001000000100000010000000000000010000000000000  
10000100000001000000000000000000001000000000001000000000001000000  
000000001000000000100000100000001100000000000000
```

REPRESENTING SETS OF INTEGERS AS BIT PATTERNS

This set:

```
UIntSet( {290} )
```

Is represented by this integer

1989292945639146568621528992587283360401824603189390869761855
907572637988050133502132224

Which has this bit pattern:

[illegible]

REPRESENTING SETS OF INTEGERS AS BIT PATTERNS (2)

UintSet() → 0

0

$$\text{UIntSet}(\{0\}) \rightarrow 1$$

1

$$\text{UIntSet}(\{1\}) \rightarrow 2$$

1	0
---	---

```
UIntSet({0, 1, 2, 4, 8}) → 279
```

$$|1|0|0|0|1|0|1|1|1|$$

```
UIntSet({0, 1, 2, 3, 4, 5, 6, 7, 8, 9}) → 1023
```

| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

UIntSet({10}) → 1024

[illegible]

UIntSet({0, 2, 4, 6, 8, 10, 12, 14, 16, 18}) → 349525

[illegible]

UIntSet({1, 3, 5, 7, 9, 11, 13, 15, 17, 19}) → 699050

1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0

SAMPLE METHOD: INTERSECTION OPERATOR &

```
76     def __and__(self, other):
77         cls = self.__class__
78         if isinstance(other, cls):
79             res = cls()
80             res._bits = self._bits & other._bits
81             return res
82         return NotImplemented
```

SAMPLE METHOD: INTERSECTION WITH ITERABLES

```
84     def intersection(self, *others):
85         cls = self.__class__
86         res = cls()
87         res._bits = self._bits
88         for other in others:
89             if isinstance(other, cls):
90                 res._bits &= other._bits
91             try:
92                 second = cls(other)
93             except TypeError:
94                 raise TypeError(INVALID_ITER_ARG_MSG)
95             else:
96                 res._bits &= second._bits
97         return res
```

DIVE INTO THE CODE

<https://github.com/standupdev/uintset>

CONCLUSION

KEY TAKEAWAYS

1. Set operations allow simpler, faster solutions for many tasks.
2. Python's set classes are lessons in idiomatic API design.
3. A set class provides good context for operator overloading.

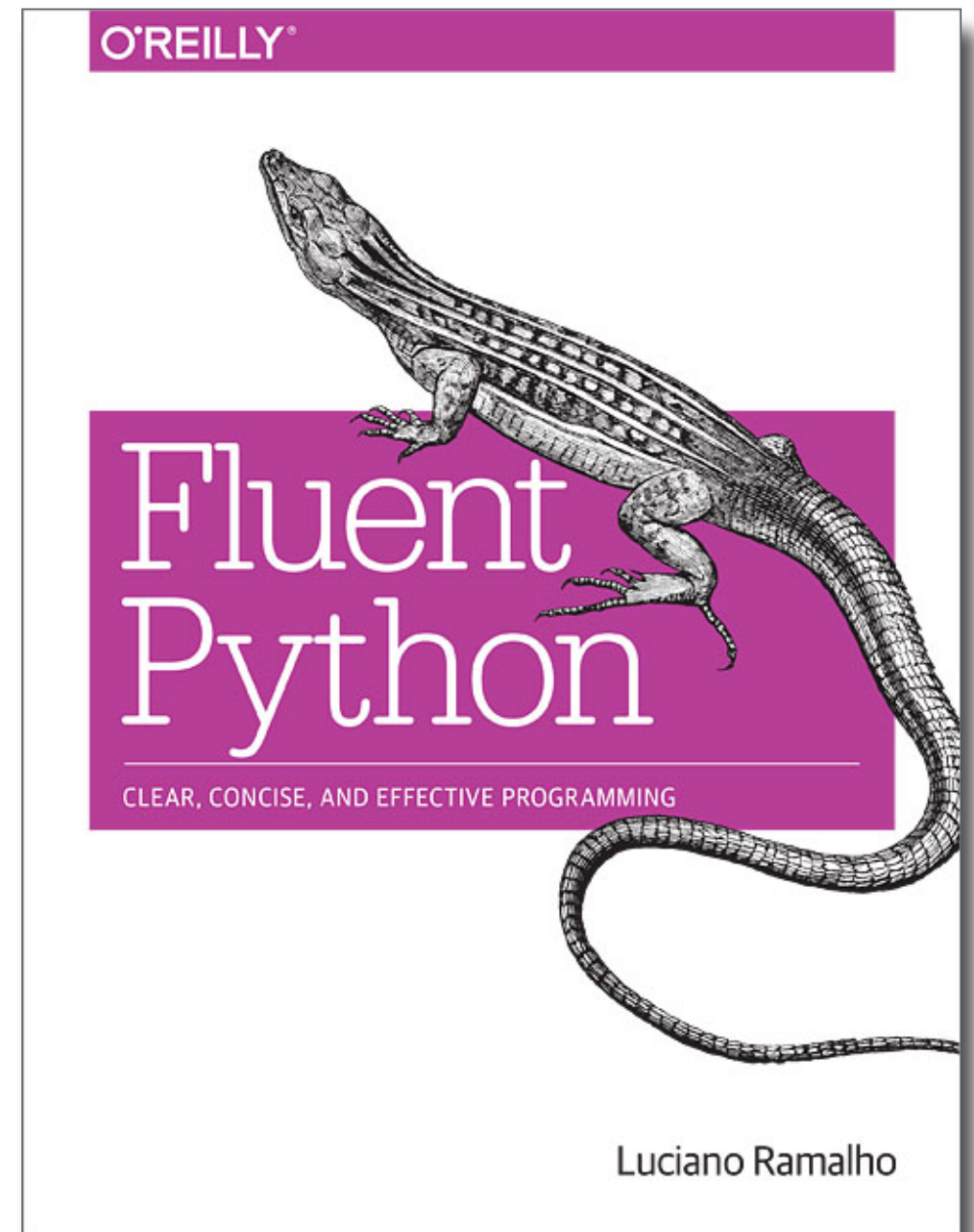
THANK YOU! COME SEE ME AT THE EXPO HALL

A deeper look at the code for UintSet

- Today, 11:45 at the JetBrains/PyCharm booth

Fluent Python book signing —*handing out free copies!*

- Today, 4:00 at the O'Reilly booth



THANK YOU!

Luciano Ramalho

@ramalhoorg | @standupdev

luciano.ramalho@thoughtworks.com

ThoughtWorks®